KUKU · AI GENERALIST ASSIGNMENT · PROBLEM 1

Find the Promos: screenplay in, ranked promo list out

Praveen Roj · Enrolment No. 23117103 · Mechanical Engineering, IIT Roorkee · praveen_r@me.iitr.ac.in

Prototype	https://kuku-assignment.vercel.app/ · upload a screenplay, get ranked promos; connect any model from "AI Model"
Code	https://github.com/iroj-09/KUKU-Assignment · pipeline, review app, tests, run steps in README
Raw outputs	Drive folder: `raw_outputs/` (unedited run on all 50 episodes) and `ranked_promos.csv` (the 60 promos, every score)

In one line: for each episode the tool finds the 20–60 s stretches an editor would most want to look at first, says *why* (hook, cliffhanger), and proves every timestamp it quotes. It shortlists; the editor still watches and cuts.

50	53,182	60	12	46 / 14 / 0
episodes in the pack	windows built in 16 s	promos, all 50 episodes covered	model requests for the whole show	timestamp PASS / PARTIAL / FAIL

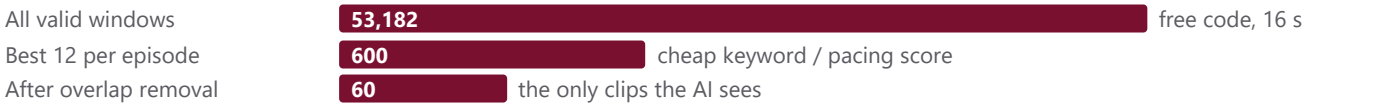
1 Why Problem 1, and how I scoped it

It is a weekly bottleneck (someone watches every episode of every new show), it comes with real data (50 episodes), and the output is checkable: an episode, start/end times, a hook and a cliffhanger I can verify against the text. I scoped it to **one step of the editor's work: a ranked shortlist of candidate promos**. Deciding where to look first is automated; watching and cutting stays human.

2 Assumptions (the brief was silent)

- Screenplay timestamps are my only evidence. "First 3 s / last 5 s" are screenplay seconds; I cannot judge acting, visuals or music.
- A promo is **one continuous 20–60 s stretch that starts and ends on event boundaries**, so no line is cut in half. Joined or cross-episode promos are not built (the output already stores a list of segments, so they can be added).
- 12 candidates per episode reach de-duplication; at most 3 survive; two clips "overlap" when the shared time is at least half of the shorter one.
- Model score weights (hook .25, cliffhanger .25, emotion .20, stand-alone .15, progression .15) and the 70% model / 30% heuristic blend are my judgement, not learned from data.

3 How it works: code, AI, human



Code	Parse → build every valid window → cheap score → keep best 12 per episode → drop overlaps → cut each clip's first 3 s and last 5 s as evidence → **re-check every timestamp the model returns** against those windows → rank. Deterministic and free.
AI (one job)	For the 60 shortlisted clips only: find the hook and cliffhanger *inside the evidence windows*, score hook / cliffhanger / emotion / stand-alone clarity / progression, write one line on why it works, list risks. Output is schema-validated; an out-of-window or missing timestamp is never trusted and downgrades the clip to PARTIAL/FAIL.
Human	Review page in the app: approve / reject / notes per clip, backup and import. Every shortlisted clip is a candidate for a person to confirm.

Re-runnable: `python run_all.py --input <folder>` (8 steps; `--no-llm` works without a key). **Model is swappable** with environment variables (Gemini, any OpenAI-compatible API, Anthropic); a test runs the whole evaluator against a fake non-Gemini server. The live app keeps the API key on the server and shows no screenplay data (confidential); a new upload uses a lightweight in-browser heuristic (3 clips per script) plus the optional AI pass.

4 Result on the 50-episode pack

60 promos covering all 50 episodes. Top ranked: Ep 32 (107.0–150.8 s: *"And what is wrong with that?!"* → *"...let's see how brave you are when it touches your skin!"*), Ep 24 (33.8–89.8 s), Ep 45 (42.0–88.2 s). Each promo carries *why it works* and *why ranked here* (score breakdown, timestamp status, main risk) in `ranked_promos.csv`.

5 Scale and cost: every show we make

The expensive judgement is spent on about 0.1% of windows. A whole 50-episode show took **12 requests, about 68k input + 13k output tokens**, so the free tier was enough, and cost grows with the shortlist, not with episode length. For many shows: one run per show (16 s of CPU), results cached by clip id (resumable), and a stronger model can re-judge only each show's top ~20. With paid API budget I would add a pairwise "which would you post first?" pass on those 20.

6 How I know it is good, and where it fails

Measured by `quality_report.md` on the real run, not hand-picked examples. These numbers come from the pipeline itself (no manual accuracy check), so they show where the tool is weak, not how often it is right.

Caught my own wrong first version. Fixed 5 s steps gave mid-line cuts and 46% of clips ≥55 s. Rebuilt on event boundaries: long clips 18%, mean model score 70.6 → 75.7, no FAIL.

Timestamps are verified, not trusted: 46 PASS, 14 PARTIAL, 0 FAIL.

14 of 60 are PARTIAL: the model quoted a strong line outside the first 3 s / last 5 s (10× cliffhanger, 4× hook).

Little choice: 40 of 50 episodes get only 1 promo (clips from one scene overlap and are pruned).

Heuristic barely agrees with the model (correlation 0.08), so its 30% share is mostly noise. It is one setting (`LOCAL_WEIGHT`) to change once labels exist.

The model is a lenient judge: 58 of 60 "promising", scores spread only ±7.6, so neighbouring ranks are not meaningfully different.

Needs earlier episodes: 12 of 60 list this as a risk; mean stand-alone score falls from 8.2 (early) to 7.6 (late).

7 AI tools I used, and where they got it wrong

Claude Code (built and refactored the pipeline, wrote tests) and **Gemini 3.5 Flash-Lite** (the judge inside the pipeline). **Where it was wrong:** AI-written docs and my earlier results claimed "beat-aligned windows", but the saved outputs came from 5-second sliding windows. Nothing flagged it; I noticed only when a fresh run shared **0 of 74** clips with the old one. Fix: regenerate everything, archive the old run, add tests for the parameters that drifted.

8 Next

Label ~50 clips with an editor and fit the weights; build joined and cross-episode promos; choose shortlists for variety so each episode offers 2–3 options; add video cues (acting, pacing) once available.